Introducción a Python (II)

GIARA

Departamento de Automática y Computación

UPNA

Índice

- Estructuras de datos avanzadas
  - Listas
  - Tuplas
  - Conjuntos (sets y frozensets)
  - Diccionarios

Listas

- Hasta ahora
  - Tipos simples números
  - Strings
  - Todos ellos eran inmutables
    - Números: al cambiar el valor de una variable numérica estamos cambiando la variable en sí misma
    - Strings: error al intentar hacer
      - s="Hola"
      - s[0]="h"

Listas

- Estructuras de datos
  - útiles para trabajar con múltiples variables del mismo o distinto tipo
  - Mutables
    - Listas
    - Conjuntos
  - Inmutables
    - Tuplas

Listas

- Dato estructurado que
  - puede almacenar cualquier número de ítems
  - los ítems pueden ser de cualquier tipo
    - Puede no ser la mejor idea tener listas de distintos tipos
  - Similar al concepto “array”
    - No necesitamos definir previamente la longitud
    - No necesitamos definir un único tipo

Listas: creación y acceso

- ¿Cómo definir una lista?
  - x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
  - type(x)
    - list
  - x = [] -> Lista vacía
    - len(x) -> 0
- ¿Cómo acceder a cada elemento de la lista?
  - x[0]
  - x[1]
  - …
  - x[len(x)-1]

Listas: creación y acceso

- ¿Existe otra forma de acceder?
  - Podemos utilizar índices negativos
  - x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    - x[-1] -> 10
    - x[-2] -> 9
    - ...
    - x[-len(x)] -> 1

Listas: creación y acceso

- Modificación de valores
  - lista [índice] = nuevo_valor
- ¡Cuidado al añadir un nuevo valor!
  - len(x) -> 10
  - x[10] = 11 -> Error!

Listas: creación y acceso

- Otras observaciones
  - x = [5]
  - y = 5
  - x == y -> False
    - type(x) -> list
    - type(y) -> int

Listas: creación y acceso

- Listas como elementos iterables
  - Como hemos visto en la presentación anterior, podemos utilizar listas como elementos iterables en un bucle for
  - for i in [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:

    print i
  - for i in x:

    print i

Listas: creación y acceso

- Persistencia de variables

  for i in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

  print i

  print i -> 10

  - La variable i sigue existiendo (no se crea de manera local al bucle)
    - del i

Listas: creación y acceso

- Podemos tener listas de cualquier tipo
  - x = [1, 2, 3, 4]
  - x=[“Antonio”, “Pepe”, “María”]
  - x=[1, 2.5, 6]
    - type(x) -> list
    - type(x[0]) -> int
    - type(x[1]) -> float

Listas: creación y acceso

- Podemos tener listas de cualquier tipo
  - x=[1, [2, 3], 4]
    - type(x) -> list
    - len(x) -> 3
    - type(x[0]) -> int
    - type(x[1]) -> list

Listas: operaciones

- Concatenación de listas: +
  - x = [1, 2, 3, 4, 5]
  - y = [6, 7, 8, 9, 10]
  - z = x + y
    - z=[1, 2, 3, 4, 5,6, 7, 8, 9, 10]
  - len(z) -> 10
- Concatenación de listas: *
  - x = [1,2, 3]
  - y = 3*x -> y=[1,2,3,1,2,3,1,2,3]

Listas: operaciones

- Formas de acceder a los índices
  - x = [5, 1, 4, 2, 7, 3, 3, 1]
  - x[2:5] -> [4, 2, 7]
    - type(x[1:1]) -> list
  - x[:6] -> [5, 1, 4, 2, 7, 3]
    - Similar a x[0:6]
  - x[6:] -> [3, 1]
    - Similar a x[6:len(x)-1]
  - x[:] -> [5, 1, 4, 2, 7, 3, 3, 1]
    - Similar a x[0:len(x)-1]

Listas: operaciones

- Modificación de elementos (las listas son mutables)
  - x=[1, 2, 3, 4, 5]
    - x[0] = 2
      - x -> [2, 2, 3, 4, 5]
    - x[0:2] = [2, 1]
      - x -> [2, 1, 3, 4, 5]
    - x[0:2] = []
      - x -> [3,4,5]
    - x[0:0] = [1, 1]
      - x-> [1, 1, 2, 3, 4, 5]
    - x[0] = [1,1] CUIDADO
      - x -> [[1,1], 2, 3, 4, 5]

Listas: operaciones

- Modificación de elementos
  - x=[1, 2, 3, 4, 5]
    - del x[0]
      - x-> [2, 3, 4, 5]
    - del x[0:2]
      - x -> [3, 4, 5]

Listas: operaciones

- Otros métodos para insertar/eliminar elementos (como una pila)
  - x = [1,2,3]
  - x.append(4)
    - x -> [1,2,3,4]
  - x = [1,2,3,4]
    - y = x.pop()
      - y -> 4
      - x -> [1,2,3]

Listas: operaciones

- Algunas funciones sencillas
  - len -> longitud
  - max/min
    - max([1,2,3]) -> 3
    - max([1,2,3],[4,2,1]) -> [4,2,1]
  - sum

Listas: operaciones

x.append(elem)	añade elem al final de x
x.count(elem)	Cuántas veces aparece elem en x
x.extend(lista)	añade la lista al final de x
x.remove(elem)	elimina la primera aparición de elem en x
x.pop([i])	devuelve y elimina la última posición de x (o la posición i-ésima)
x.index(elem)	devuelve la posición de la primera aparición de elem en x
x.insert(i,elem)	inserta elem antes de la i-ésima posición
x.remove(elem)	elimina la primera aparición de elem en x
x.sort	ordena los elementos de x
x.reverse	da la vuelta a los elementos de x

Listas por comprensión

- Permiten una forma concisa de crear listas
- Ejemplo:

```
x = [1, 2, 3, 4, 5]
cuadrados = []
for i in x:
    cuadrados.append(i**2)
```

- Por comprensión

```
cuadrados = [i**2 for i in x]
```

- Incluso para inicializar vectors

```
vector = [0 for i in range(10)]
```

Listas por comprensión

- Podemos añadir condiciones a la construcción

```
x = range(100)
cuadrados = [i**2 for i in x  if i%3 == 0]
cuadrados -> [0, 9, 36, 81, ...]
```

- https://docs.python.org/2/tutorial/datastructures.html#list-comprehensions

dobles listas

- Antes hemos visto que podíamos tener
  - x = [1, [2, 3], 4]
    - len(x) -> 3
  - x[1] -> [2,3]
  - x[1][0] -> 2
  - x[1][1] -> 3

dobles listas

- Muy útil para la representación de matrices

  M = [[1, 2, 3],

  [4, 5, 6],

  [7, 8, 9]]

  - Acceso a las filas
    - M[0],M[1],M[2]
  - Acceso a elementos concretos
    - M[0][2] -> 3

dobles listas

- Matrices por comprensión

```
M = [ [0 for i in range(5)] for j in range(5)]

M = [ [i+j for i in range(5)] for j in range(5)]

    M = []
    for i in range(0,5):

        fila = []
        for j in range(0,5):

            fila.append(i+j)

        M.append(fila)
```

Listas y memoria

- Cuidado al asignar/copiar listas
  - Comportamiento esperado

  x = [1, 2, 3]

  y = [1, 2, 3]

  x[0] = 2

  print x -> [2, 2, 3]

  print y -> [1, 2, 3]

Listas y memoria

- Cuidado al asignar/copiar listas
  - Comportamiento esperado

  x = [1, 2, 3]

  y = x

  x[0] = 2

  print x -> [2, 2, 3]

  print y -> [2, 2, 3]

  - En realidad, la asignación y = x hace que x e y sean exactamente el mismo objeto
- Clonado de listas
  - b = a[:]

Listas y memoria

- Función id(variable) nos devuelve el identificador de dicha variable (como una posición en memoria)

  x = [1, 2, 3]; id(x) -> 188692040L

  y = [1, 2, 3]; id(y) -> 188600392L

  x = [1, 2, 3]; id(x) -> 188692040L

  y = x; id(y) -> 188692040L

  x = [1, 2, 3]; id(x) -> 188692040L

  y = x[:]; id(y) -> 188600392L

Tuplas

- Tupla: lista de elementos separados por coma

  t = ‘a’, ‘e’, ‘i’, ‘o’, ‘u’

  type(t) -> tuple

  - Creación de una tupla vacía

  t = ()

  - Creación de una tupla de un solo elemento

  t = ‘a’,

    - Para especificar que t no es un string o un número, etc.

- Tuplas a partir de listas
  - l = [1,2,3]
  - t = tuple(l)

Tuplas

- ¿Y en qué se distingue de las listas?
  - t = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
  - t[0] = 2 -> Error
- Las tuplas son inmutables
  - No pueden ser modificadas
  - Deben ser reasignadas a una tupla nueva
  - Únicamente podemos cambiar algún elemento de la tupla si éste es mutable
    - t = [1, 2, 3], [4, 5 6]
    - t[0][0] = 2
    - t -> ([2, 2, 3], [4, 5, 6])

Tuplas

- Tuplas para asignaciones múltiples
  - x, y = 5, 6
    - x -> 5
    - y -> 6
  - x, y = 7 -> Error
  - x, y = 5, 6, 7 -> Error
  - x, y = 5, 6
  - x, y = y, x
    - x -> 6
    - y -> 5

Tuplas

- Tuplas para asignaciones múltiples: packing/unpacking
  - t = 1.0, ‘a’, 23
    - t = (1.0, ‘a’, 23) -> packing
  - var1, var2, var3 = t
    - var1= 1.0
    - var2 = ‘a’
    - var3 = 23 -> unpacking

Conjuntos

- Estructura para almacenar un conjunto de elementos no repetidos sin ordenación
  - s = set() -> conjunto vació
  - s = set([1, 2, 3, 4, 3, 2, 1])
    - s -> {1, 2, 3, 4}
  - s = set('Hola mundo')
    - s -> {' ', 'H', 'a', 'd', 'l', 'm', 'n', 'o', 'u'}
- En general, podemos almacenar únicamente tipos inmutables

Conjuntos

- Operaciones conjuntistas
  - x = {‘a’, ‘e’, ‘i’, ‘o’, ‘u’}
  - y = {‘a’, ‘b’, ‘c’, ‘d’, ‘e’}
  - unión: |
    - x | y -> {'a', 'b', 'c', 'd', 'e', 'i', 'o', 'u'}
  - intersección
    - x & y -> {'a', 'e'}
  - Diferencia
    - x-y -> {'i', 'o', 'u'}
  - Diferencia simétrica
    - x ^y -> {'b', 'c', 'd', 'i', 'o', 'u'}

Conjuntos

- Al no tener orden no podemos hacer operaciones de indexación, etc.
  - x in s
  - len(s)
  - s.add(elem)
  - s.remove(elem)
- Es mutable!
- https://docs.python.org/2/library/stdtypes.html#set

Frozenset

- Frozenset es la versión inmutable de los sets
  - x = set(‘Hola’)
    - x -> {'H', 'a', 'l', 'o'}
  - x = frozenset(x)
    - x -> frozenset({'H', 'a', 'l', 'o'})
- A diferencia del set, los frozensets son inmutables y por tanto son hashables
- https://docs.python.org/2/library/stdtypes.html?highlight=frozenset#frozenset

Diccionarios

- En general, estamos acostumbrados a estructuras indexadas mediante números enteros
  - Algunos lenguajes permiten también la indexación mediante caracteres, etc.
- Un diccionario es un tipo de dato estructurado en el que los índices de los elementos pueden ser casi cualquier tipo
  - Similar a una memoria asociativa
- Distinguimos en un diccionario
  - Clave: en general, cualquier tipo inmutable
  - Valor

Diccionarios

- Creación de diccionarios
  - Diccionario vacío
    - d = dict()
  - Asignación de un ítem clave-valor
    - d[‘hola’] = 1
      - d -> {'hola': 1}
    - d['estas'] = 1
      - d -> {'estas': 1, 'hola': 1}

Diccionarios

- Creación de diccionarios
  - d = dict([(key1, val1), (key2, val2), ..., (keyn,valn)])
  - d = dict(key1=val1, key2=val2, ..., keyn=valn)
    - Cuando key1,key2,...,keyn son strings
  - Por comprensión
    - {x : x**2 for x in range(5)}
    - {x: 0 for x in range(5)}
    - {x:i for x,i in zip('hola',range(4))}
      - la función zip toma n listas y devuelve una lista de tuplas, en las que la i-ésima tuple contiene los i-ésimos elementos de las tuplas de entrada

Diccionarios

- Acceso a los elementos: mediante las claves
  - d[‘hola’]
  - ‘hola’ in d -> True
  - ‘estas’ in d -> True
  - 1 in d -> False
  - d.values: devuelve una lista de los valores
    - d.values() -> [1, 1]
  - d.keys: devuelve una lista de claves
    - d.keys() -> [‘estas’, ‘hola’]
  - d.items(): devuelve una lista de tuplas clave-valor
    - d.items() -> [('estas', 1), ('hola', 1)]

Diccionarios

- Acceso a los elementos: iterando
  - d=dict(a=0,b=1,c=2,d=3)

```
for x in d:
    print x
```

```
'a'
'b'
'c'
'd'
```

```
for x,i in d.items():
    print "{}:{}".format(x,i)
```

```
'a':0
'b':1
'c':2
'd':3
```

Diccionarios

- Algunas built-in functions:
  - len -> número de ítems
  - max/min -> máxima/mínima clave
  - sum -> suma de las claves si son numéricas

Diccionarios

- d = dict(a=1, b=2, c=3, d=4)
  - d.get(‘a’,0)
    - 1
  - d.get(‘k’,0)
    - 0
  - d.clear()
    - d -> {}
  - d.pop(‘a’)
    - elimina el ítem (‘a’,1) y devuelve el valor asociado a ‘a’
    - d = {'b': 2, 'c': 3, 'd': 4}
  - d.del(‘b’)
    - {'c': 3, 'd': 4}